

Estudo Completo — CyberProject IoT

Análise linha a linha do sistema completo: ESP8266 + Flask API + MySQL + Google Sheets

```
pip install Flask mysql-connector-python flask-cors Werkzeug gspread google-auth
```

Cada pacote instalado tem uma função específica:

Biblioteca	Função no projeto
Flask	Framework web que cria o servidor HTTP em Python. Gerencia rotas, requisições e respostas
mysql-connector-python	Driver oficial da Oracle para conectar Python ao banco de dados MySQL
flask-cors	Habilita CORS (Cross-Origin Resource Sharing) — permite que o ESP8266 (outro IP) envie dados para a API sem ser bloqueado
Werkzeug	Utilitário do Flask. Aqui é usado especificamente para generate_password_hash e check_password_hash
gspread	Biblioteca para manipular Google Sheets via Python
google-auth	Autentica a aplicação nos serviços do Google usando credenciais de conta de serviço

Perguntas comuns de professor:

"Por que precisamos do flask-cors?" → Porque o ESP8266 faz requisições de um IP diferente. Sem CORS, o navegador/dispositivo bloquearia a requisição por política de segurança de origem.

"O que é Werkzeug?" → É a biblioteca de utilitários WSGI na qual o Flask é construído. Fornece ferramentas de segurança como hash de senhas.

"O que é WSGI?" → Web Server Gateway Interface — padrão de comunicação entre servidores web e aplicações Python.

SESSÃO 2 — Bibliotecas do Arduino (ESP8266)

```
#include <ESP8266WiFi.h>           // (1)
#include <ESP8266WebServer.h>       // (2)
#include <ESP8266HTTPClient.h>     // (3)
#include <WiFiClient.h>             // (4)
#include <ArduinoJson.h>            // (5)
#include <DHT.h>                    // (6)
```

(1) ESP8266WiFi.h Biblioteca principal do módulo WiFi do ESP8266. Permite conectar à rede, verificar status, obter IP local. Sem ela, o ESP não tem acesso à rede.

(2) ESP8266WebServer.h Cria um servidor HTTP diretamente no ESP8266 na porta 80. O objeto `server` usado no código vem daqui. Permite que o ESP sirva páginas web ou responda a requisições de outros dispositivos na rede.

(3) ESP8266HTTPClient.h Permite que o ESP8266 atue como cliente HTTP — ou seja, ele que faz as requisições `GET` e `POST` para o servidor externo (a API Flask).

(4) WiFiClient.h Gerencia a conexão TCP de baixo nível. O `HTTPClient` usa um `WiFiClient` internamente para abrir o socket de comunicação. Necessário para definir timeout da conexão (`client.setTimeout(3000)`).

(5) ArduinoJson.h Serializa e desserializa JSON no Arduino. Permite criar objetos JSON para enviar à API e ler respostas JSON (como a data/hora do servidor).

(6) DHT.h Driver do sensor DHT11/DHT22. Fornece os métodos `dht.readTemperature()` e `dht.readHumidity()`.

Perguntas comuns de professor:

"Qual a diferença entre ESP8266HTTPClient e ESP8266WebServer?" → O WebServer faz o ESP responder requisições (ele é o servidor). O HTTPClient faz o ESP enviar requisições para outro servidor (ele é o cliente).

"Por que usar ArduinoJson e não montar a string JSON manualmente?" → Montagem manual é propensa a erros de formatação (aspas, vírgulas). O ArduinoJson garante JSON válido e facilita a leitura da resposta.

"O que é DynamicJsonDocument?" → É o objeto que aloca o documento JSON na heap (memória dinâmica). O número 256 é o tamanho em bytes reservado para o documento.

SESSÃO 3 — Código Arduino (ESP8266) linha a linha

Definição de pinos e constantes

```
#define BOTAO_WIFI_ON D0 // Pino D0: botão para CONECTAR ao WiFi
#define BOTAO_WIFI_OFF D1 // Pino D1: botão para DESCONECTAR do WiFi
#define BUZZER D5 // Pino D5: saída para o buzzer (alerta sonoro)
#define DHTPIN D2 // Pino D2: entrada de dados do sensor DHT11
#define DHTTYPE DHT11 // Define o modelo do sensor (DHT11 ou DHT22)
```

`#define` é uma diretiva de pré-processador — não ocupa memória como uma variável, é substituído diretamente no código antes da compilação.

=====

```
const char* WIFI_SSID = "Cyber-Projeto"; // Nome da rede WiFi (SSID)
const char* WIFI_PASS = "Senai@122"; // Senha da rede WiFi
```

`const char*` é um ponteiro para string constante em C++. Usado ao invés de `String` para economizar memória RAM no microcontrolador.

```
String servidor = "ipconfig"; // IP da máquina onde roda a API Flask
const String AUTH_TOKEN = "CyberProject"; // Token de autenticação enviado no
header HTTP
```

O IP deve ser fixo (ou reservado no roteador) para o ESP sempre encontrar a API.

====

```
String nomeSensor = "Temperatura Sala"; // Identificação do sensor — usado no log
Serial
```

```
DHT dht(DHTPIN, DHTTYPE); // Cria objeto DHT no pino D2, tipo DHT11
ESP8266WebServer server(80); // Cria objeto servidor HTTP na porta 80
```

```
bool wifiAtivo = false; // Flag: controla se o WiFi está conectado
unsigned long ultimoEnvio = 0; // Armazena o timestamp do último envio
(milissegundos)
```

`unsigned long` é necessário porque `millis()` retorna o tempo em ms desde o boot — pode chegar a valores muito grandes. `int` transbordaria.

Função obterEstado()

String obterEstado(float temperatura) {

```
    if (isnan(temperatura))    // isnan() = "is not a number"
        return "FALHA_SENSOR";    // Sensor desconectado ou com defeito retorna NaN
```

```
    if (temperatura >= 40)
        return "CRITICO";    // Acima de 40°C: estado crítico
```

```
    if (temperatura >= 30)
        return "ALERTA";    // Entre 30°C e 39.9°C: alerta
```

```
    return "NORMAL";    // Abaixo de 30°C: normal
}
```

Por que usar `isnan()`? O DHT11 retorna `NaN` (Not a Number) quando a leitura falha. Operar matematicamente com `NaN` causa comportamento imprevisível.

====

```
Função obterDataHoraServidor()  
String obterDataHoraServidor() {
```

====

```
    if (WiFi.status() != WL_CONNECTED) // Verifica conexão antes de tentar  
        return "--";  
  
    WiFiClient client;  
    HTTPClient http;  
  
    client.setTimeout(2000);           // Timeout TCP: 2 segundos para conectar  
  
    String url = "http://" + servidor + ":5000/hora"; // Monta URL do endpoint  
  
    if (!http.begin(client, url))      // Inicia conexão HTTP. Se falhar, retorna "--"  
        return "--";  
  
    http.setTimeout(2000);             // Timeout HTTP: 2 segundos para receber resposta  
  
    int httpCode = http.GET();          // Faz a requisição GET e armazena o código HTTP  
  
    if (httpCode == 200) {              // 200 = OK (sucesso)  
  
        String payload = http.getString(); // Lê o corpo da resposta (JSON com a data/hora)  
  
        DynamicJsonDocument doc(256);    // Cria documento JSON com 256 bytes  
  
        if (!deserializeJson(doc, payload)) { // Converte string JSON para objeto  
  
            String dataHora = doc["datahora"].as<String>(); // Extrai o campo "datahora"  
  
            http.end();                  // Libera recursos da conexão HTTP  
  
            return dataHora;  
        }  
    }  
  
    http.end();  
    return "--";  
}
```

Função enviarDadosAPI()

```
void enviarDadosAPI(float temperatura, float umidade) {
```

```
    if (WiFi.status() != WL_CONNECTED) // Guarda de segurança
        return;
```

```
    String estado = obterEstado(temperatura); // Determina estado baseado na temperatura
```

```
    DynamicJsonDocument doc(256); // Documento JSON para o corpo da requisição
```

```
    doc["temperatura"] = isnan(temperatura) ? 0.0 : temperatura; // Evita enviar NaN
```

```
    doc["umidade"] = isnan(umidade) ? 0.0 : umidade;
```

```
    doc["rotacao"] = 0; // Rotação fixa em 0 (sensor não implementado)
```

```
    doc["estado"] = estado;
```

```
    String jsonBody;
```

```
    serializeJson(doc, jsonBody); // Converte o objeto JSON para string
```

```
    WiFiClient client;
```

```
    HTTPClient http;
```

```
    client.setTimeout(3000);
```

```
    String url = "http://" + servidor + ":5000/receive_data"; // Endpoint de recebimento
```

```
    if (!http.begin(client, url))
```

```
        return;
```

```
    http.setTimeout(3000);
```

```
    http.addHeader("Content-Type", "application/json"); // Informa que o body é JSON
```

```
    http.addHeader("Authorization", AUTH_TOKEN); // Token de segurança
```

```
    int httpCode = http.POST(jsonBody); // Envia requisição POST com o JSON
```

```
    Serial.print("HTTP: ");
```

```
    Serial.println(httpCode); // Loga o código de resposta (201 = Created)
```

```
    http.end();
```

```
}
```

Pergunta importante: *"Por que `isnan(temperatura) ? 0.0 : temperatura?`"*
Operador ternário — se temperatura for NaN, envia 0.0. JSON não suporta NaN nativamente; enviá-lo quebraria o parser da API.

====

Funções `conectarWiFi()` e `desconectarWiFi()`

```
void conectarWiFi() {

    WiFi.begin(WIFI_SSID, WIFI_PASS); // Inicia processo de conexão

    unsigned long inicio = millis(); // Marca o tempo de início

    while (millis() - inicio < 15000) { // Tenta por até 15 segundos

        yield(); // Cede tempo ao sistema operacional do ESP8266
                // CRÍTICO: sem yield(), o watchdog reseta o ESP

        if (WiFi.status() == WL_CONNECTED) {
            wifiAtivo = true;
            server.begin(); // Só inicia o servidor web após conectar
            return;
        }

        delay(500);
        Serial.print(".");
    }

    WiFi.disconnect(); // Se não conectou, limpa o estado
}

=====
void desconectarWiFi() {

    WiFi.disconnect(); // Encerra a conexão WiFi
    server.stop(); // Para o servidor HTTP
    wifiAtivo = false; // Atualiza a flag de controle
}
```

Por que `yield()` é crítico? O ESP8266 tem um watchdog timer. Se o processador ficar preso em um loop por mais de ~1 segundo sem ceder controle ao sistema, o ESP reinicia automaticamente. `yield()` ou `delay()` evitam esse reset.

```
setup() e loop()
void setup() {
  Serial.begin(115200);          // Inicia comunicação serial a 115200
  baud

  pinMode(BOTAO_WIFI_ON, INPUT_PULLUP); // Entrada com
  resistor pull-up interno
  pinMode(BOTAO_WIFI_OFF, INPUT_PULLUP); // Em repouso: HIGH.
  Pressionado: LOW
  pinMode(BUZZER, OUTPUT);      // Saída digital para o buzzer

  dht.begin();                  // Inicializa o sensor DHT
}
```

Por que INPUT_PULLUP? Sem o resistor pull-up, o pino flutuaria entre HIGH e LOW aleatoriamente quando não pressionado, gerando leituras falsas. Com pull-up interno, o pino fica estável em HIGH até ser pressionado (vai a LOW).

```
void loop() {

    // Botão D0 pressionado E WiFi desligado → conecta
    if (digitalRead(BOTAO_WIFI_ON) == LOW && !wifiAtivo) {
        conectarWiFi();
        delay(1000); // Debounce: evita múltiplos acionamentos
    }
    -----
    // Botão D1 pressionado E WiFi ligado → desconecta
    if (digitalRead(BOTAO_WIFI_OFF) == LOW && wifiAtivo) {
        desconectarWiFi();
        delay(1000);
    }
    -----
    // A cada 20 segundos (20000ms), lê o sensor
    if (millis() - ultimoEnvio > 20000) {

        float temperatura = dht.readTemperature(); // Lê temperatura em Celsius
        float umidade     = dht.readHumidity();    // Lê umidade relativa em %

        // [logs Serial omitidos para brevidade]
        -----
        // Alerta sonoro: buzzer toca se temperatura < 27°C
        if (!isnan(temperatura) && temperatura < 27) {
            digitalWrite(BUZZER, HIGH); // Liga o buzzer
            delay(200);                  // Mantém ligado por 200ms
            digitalWrite(BUZZER, LOW);  // Desliga
        }
        -----
        if (wifiAtivo) {
            enviarDadosAPI(temperatura, umidade);
        }
        -----
        ultimoEnvio = millis(); // Atualiza o timestamp para contar os próximos 20s
    }
    -----
    if (wifiAtivo) {
        server.handleClient(); // Processa requisições pendentes no servidor HTTP interno
    }
}
```

Pergunta clássica: "*Por que `millis()` - `ultimoEnvio` > 20000 e não `delay(20000)`?*" `delay()` bloqueia o processador inteiramente — os botões seriam

ignorados e o servidor HTTP pararia de responder. Com `millis()`, o loop continua rodando e verificando tudo a cada ciclo.

SESSÃO 4 — Código Python (Flask API) linha a linha

Imports

```
import mysql.connector # Driver MySQL para Python
from flask import (
```

```
    Flask, # Classe principal da aplicação web
    request, # Objeto com dados da requisição recebida (body, headers)
    jsonify, # Converte dicionário Python → resposta JSON com headers corretos
    render_template, # Renderiza arquivos HTML da pasta /templates
    redirect, # Redireciona para outra URL
    url_for # Gera URLs dinamicamente pelo nome da função (boa prática))
```

```
from flask_cors import CORS # Habilita CORS em todas as rotas
from werkzeug.security import (
```

```
    generate_password_hash, # Gera hash bcrypt da senha (nunca armazena texto puro)
    check_password_hash # Compara senha fornecida com o hash armazenado)
```

```
import json # Parseia JSON manualmente (usado para o body do ESP8266)
from datetime import datetime # Gera timestamps para MySQL e Google Sheets
import os # Funções do sistema operacional (caminhos de arquivo)
import gspread # Manipula planilhas Google Sheets
from google.oauth2.service_account import Credentials # Autenticação OAuth2 com conta de serviço
```

Configuração inicial

```
app = Flask(__name__)
```

Cria a instância da aplicação Flask

__name__ diz ao Flask onde encontrar templates e arquivos estáticos

CORS(app)

Aplica CORS em todas as rotas — requisições de qualquer origem são aceitas

Necessário para o ESP8266 (IP diferente) conseguir fazer POST

```
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
```

os.path.abspath(__file__) → caminho absoluto do arquivo app.py

os.path.dirname() → pasta que contém o arquivo

Resultado: garante que caminhos funcionem independente de onde o script é executado

```
CAMINHO_JSON = os.path.join(BASE_DIR, "credentials.json")
```

Monta o caminho completo para o arquivo de credenciais do Google

os.path.join() usa o separador correto para cada OS (\ no Windows, / no Linux)

```
AUTH_TOKEN = "CyberProject"
```

Token simples de autenticação — a API só aceita dados do ESP que enviar este token

Em produção, usar JWT ou tokens mais robustos

Variáveis globais do dashboard

Python

```
ultima_temperatura = "--" # Valor exibido no dashboard antes da primeira leitura
```

```
ultima_umidade = "--"
```

```
ultimo_estado = "NORMAL"
```

```
ultima_datahora = "--"
```

Estas variáveis ficam na memória RAM do servidor enquanto ele está rodando. São atualizadas a cada POST do ESP8266 e lidas pela rota `/dashboard`.

Atenção do professor: *"Qual o problema de usar variáveis globais assim?"* Se o servidor reiniciar, os dados se perdem. Em produção, isso seria lido diretamente do banco de dados.

Configuração MySQL

```
MYSQL_CONFIG = {  
    "host": "localhost",      # MySQL rodando na mesma máquina  
    "user": "ADM_login",     # Usuário criado especificamente para a aplicação  
    "password": "CyberCopilots", # Senha do usuário  
    "database": "api_automacao" # Banco de dados que será usado  
}
```

Conexão Google Sheets

```
ESCOPOS = [  
    "[googleapis.com](https://www.googleapis.com/auth/spreadsheets)", # Permissão de leitura/escrita em Sheets  
    "[googleapis.com](https://www.googleapis.com/auth/drive)"        # Permissão de acesso ao Drive (para abrir por URL)  
]
```

try:

```
credenciais = Credentials.from_service_account_file(  
    CAMINHO_JSON,      # Carrega o arquivo credentials.json  
    scopes=ESCOPOS     # Define o que a conta de serviço pode acessar  
)  
  
cliente = gspread.authorize(credenciais) # Autentica e cria o cliente gspread  
  
URL_PLANILHA = "[docs.google.com](https://docs.google.com/spreadsheets/d/)"  
  
planilha = cliente.open_by_url(URL_PLANILHA) # Abre a planilha pela URL  
  
aba = planilha.worksheet("Historico_Geral") # Seleciona a aba específica  
  
print("☑ Google Sheets conectado!")
```

except Exception as e:

```
print(f"❌ Erro Google Sheets: {e}")  
# A aplicação continua funcionando mesmo sem o Sheets  
# O dado ainda é salvo no MySQL
```

Função `validate_sensor_data()`

```
def validate_sensor_data(data):
```

```
    required = ["temperatura", "umidade", "rotacao", "estado"]
```

```
    # Lista de campos obrigatórios no JSON
```

```
    if not isinstance(data, dict):
```

```
        return False, "JSON inválido"
```

```
    # Verifica se o JSON foi parseado corretamente (deve ser um dicionário)
```

```
    for key in required:
```

```
        if key not in data:
```

```
            return False, f"Campo ausente: {key}"
```

```
    # Verifica se todos os campos obrigatórios estão presentes
```

```
    try:
```

```
        float(data["temperatura"]) # Testa se pode ser convertido para float
```

```
        float(data["umidade"])
```

```
        int(data["rotacao"])      # Testa se pode ser convertido para int
```

```
        str(data["estado"])      # Qualquer coisa pode virar string, mas valida existência
```

```
    except (ValueError, TypeError):
```

```
        return False, "Tipos inválidos"
```

```
    # Se qualquer conversão falhar, os dados são inválidos
```

```
    return True, "" # Retorna tupla (sucesso, mensagem_erro)
```

Por que retornar uma tupla (`bool`, `str`)? Permite saber não só se falhou, mas por que falhou — facilita depuração e resposta de erro mais clara para o cliente.

Rotas de autenticação

```
@app.route("/")
def home():
    return redirect("/login")
# Redireciona a raiz para o login — boa prática de UX

-----

@app.route("/register", methods=["GET", "POST"])
def register():

    if request.method == "GET":
        return render_template("register.html")
    # GET: apenas exibe o formulário

    # POST: processa o registro
    username = request.form.get("username") # Pega o campo "username" do formulário HTML
    password = request.form.get("password")
    confirm_password = request.form.get("confirm_password")

    if password != confirm_password:
        return render_template("register.html", error="As senhas não coincidem")
    # Passa variável "error" para o template — o HTML a exibe para o usuário

    if len(password) < 6:
        return render_template("register.html", error="Senha muito curta")

    try:
        conexao = mysql.connector.connect(**MYSQL_CONFIG)
        # **MYSQL_CONFIG desempacota o dicionário como argumentos nomeados
        # Equivale a: connect(host="localhost", user="ADM_login", ...)

        cursor = conexao.cursor()

        query_check = "SELECT id FROM usuarios WHERE username = %s"
        cursor.execute(query_check, (username,))
        # %s é placeholder — NUNCA concatenar strings em SQL (risco de SQL Injection)
        # O conector escapa os valores automaticamente

        usuario_existente = cursor.fetchone()
        # fetchone() retorna uma linha ou None

        if usuario_existente:
            cursor.close()
```

```

        conexao.close()

        return render_template("register.html", error="Usuário já existe")

senha_hash = generate_password_hash(password)
# Gera hash bcrypt com salt aleatório
# Resultado: string como "$2b$12$..."
# NUNCA armazenar senha em texto puro

query_insert = """
    INSERT INTO usuarios (username, password_hash)
    VALUES (%s, %s)
    """

cursor.execute(query_insert, (username, senha_hash))

conexao.commit()
# OBRIGATÓRIO para INSERT/UPDATE/DELETE — confirma a transação no banco

cursor.close()
conexao.close()

return redirect("/login")

except Exception as e:
    return render_template("register.html", error=f"Erro no servidor: {e}")

-----

@app.route("/login", methods=["GET", "POST"])
def login():

    if request.method == "GET":
        return render_template("login.html")

    username = request.form.get("username")
    password = request.form.get("password")

    try:
        conexao = mysql.connector.connect(**MYSQL_CONFIG)

        cursor = conexao.cursor(dictionary=True)
        # dictionary=True → retorna linhas como dicionários {"campo": valor}
        # Sem isso, retorna tuplas (0, "usuario", "hash") — menos legível

```

```

query = "SELECT * FROM usuarios WHERE username = %s"
cursor.execute(query, (username,))

usuario = cursor.fetchone() # Retorna o dicionário do usuário ou None

cursor.close()
conexao.close()

if usuario:
    senha_correta = check_password_hash(
        usuario["password_hash"], # Hash armazenado no banco
        password                  # Senha digitada pelo usuário
    )
    # check_password_hash re-hash a senha com o mesmo salt e compara
    # Nunca é possível "descriptografar" — é comparação de hashes

    if senha_correta:
        return redirect("/dashboard")

return render_template("login.html", error="Usuário ou senha inválidos")
# Mensagem genérica: não informa se o usuário existe ou não (segurança)

except Exception as e:
    return render_template("login.html", error=f"Erro no servidor: {e}")

```

Rota /receive_data — coração da API

```
@app.route("/receive_data", methods=["POST"])
def receive_data():

    global ultima_temperatura, ultima_umidade, ultimo_estado, ultima_datahora
    # Declara intenção de modificar variáveis globais (obrigatório em Python)

    auth_header = request.headers.get("Authorization")
    # Lê o header "Authorization" da requisição

    if auth_header != AUTH_TOKEN:
        return jsonify({"error": "Unauthorized"}), 401
    # 401 = Unauthorized. Rejeita requisições sem token válido
    # Impede que qualquer dispositivo na rede envie dados falsos

    raw = request.get_data()
    # Lê o body bruto da requisição como bytes
    # Usado no lugar de request.json para maior controle sobre erros

    if not raw:
        return jsonify({"error": "Body vazio"}), 400
    # 400 = Bad Request

    try:
        data = json.loads(raw)
        # Converte bytes → string → dicionário Python
    except json.JSONDecodeError:
        return jsonify({"error": "JSON inválido"}), 400

    ok, msg = validate_sensor_data(data)
    # Desempacota a tupla retornada pela função de validação
    if not ok:
        return jsonify({"error": msg}), 400

    temperatura = float(data["temperatura"])
    umidade     = float(data["umidade"])
    rotacao     = int(data["rotacao"])
    estado      = str(data["estado"]).upper() # Padroniza para maiúsculas

    agora_mysql = datetime.now()
    # Objeto datetime completo para inserção no MySQL

    agora_sheets = agora_mysql.strftime("%d/%m/%Y %H:%M:%S")
    # Formata para string legível: "11/06/2026 08:27:13"

    # Atualiza variáveis globais para o dashboard
    ultima_temperatura = temperatura
    ultima_umidade     = umidade
    ultimo_estado      = estado
    ultima_datahora    = agora_sheets

    # --- MYSQL ---
    try:
        conexao = mysql.connector.connect(**MYSQL_CONFIG)
        cursor = conexao.cursor()

        query_leitura = """
            INSERT INTO leituras
            (dispositivo_id, temperatura, umidade, rotacao, estado, data_hora)
            VALUES (%s, %s, %s, %s, %s, %s)
            """

        cursor.execute(query_leitura, (1, temperatura, umidade, rotacao, estado, agora_mysql))
        conexao.commit()

        leitura_id = cursor.lastrowid
        # Retorna o ID auto_increment gerado para a linha recém-inserida
        # Usado para criar o alerta com referência à leitura (chave estrangeira)
```

```

if estado in ["ALERTA", "CRÍTICO", "CRITICO"]:
    # Verifica nas duas grafias (com e sem acento) por segurança

    query_alerta = """
    INSERT INTO alertas (leitura_id, tipo_alerta, descricao, data_hora)
    VALUES (%s, %s, %s, %s)
    """
    cursor.execute(query_alerta, (
        leitura_id,
        estado,
        f"Sistema detectou {estado}",
        agora_mysql
    ))
    conexao.commit()

cursor.close()
conexao.close()

except Exception as e:
    print(f"✘ Erro MySQL: {e}")
    # Não retorna erro para o ESP — continua para tentar salvar no Sheets

# --- GOOGLE SHEETS ---
try:
    aba.append_row([agora_sheets, temperatura, umidade, rotacao, estado])
    # Adiciona uma linha no final da planilha com os 5 valores
except Exception as e:
    print(f"✘ Erro Sheets: {e}")

return jsonify({
    "status": "success",
    "message": "Dados recebidos",
    "timestamp": agora_sheets
}), 201
# 201 = Created — indica que um recurso foi criado com sucesso (padrão REST)
-----

```

Rota /hora

```
@app.route("/hora")
```

```
def hora():
```

```
    agora = datetime.now().strftime("%d/%m/%Y %H:%M:%S")
```

```
    return jsonify({"datahora": agora})
```

```
# O ESP8266 consulta este endpoint para sincronizar o horário
```

```
# ESP8266 não tem RTC (relógio em tempo real) — usa o servidor como referência
```

Inicialização do servidor

```
if __name__ == "__main__":  
  
    # Só executa o bloco abaixo se o script for rodado diretamente  
  
    # Se importado como módulo, não inicia o servidor automaticamente  
  
    app.run(  
  
        host="0.0.0.0", # Escuta em todas as interfaces de rede (não só localhost)  
  
        # Permite que o ESP8266 acesse de outro IP na rede  
  
        port=5000,      # Porta padrão do Flask  
  
        debug=True      # Recarrega automaticamente ao salvar; mostra erros detalhados  
  
        # NUNCA usar debug=True em produção  
  
    )
```

SESSÃO 5 — Banco de Dados MySQL

```
-- Criação do banco de dados  
CREATE DATABASE api_automacao;  
  
-- Seleção do banco  
USE api_automacao;  
  
-- Tabela de usuários  
CREATE TABLE usuarios (  
    id          INT AUTO_INCREMENT PRIMARY KEY,  
    -- AUTO_INCREMENT: ID gerado automaticamente a cada insert  
    -- PRIMARY KEY: identificador único, indexado automaticamente  
  
    username    VARCHAR(100) NOT NULL UNIQUE,  
    -- VARCHAR(100): string de até 100 caracteres  
    -- NOT NULL: campo obrigatório  
    -- UNIQUE: não permite dois usuários com o mesmo nome  
  
    password_hash VARCHAR(255) NOT NULL,  
    -- Armazena o hash bcrypt (sempre ~60 caracteres, mas 255 por segurança)  
  
    criado_em    DATETIME DEFAULT CURRENT_TIMESTAMP
```

```

-- Preenchido automaticamente com a data/hora do insert
);

-- Tabela de dispositivos (ESP8266)
CREATE TABLE dispositivos (
    id      INT AUTO_INCREMENT PRIMARY KEY,
    nome     VARCHAR(100) NOT NULL,
    descricao TEXT,
    criado_em DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Insert do dispositivo padrão (dispositivo_id = 1 usado na API)
INSERT INTO dispositivos (nome, descricao)
VALUES ('ESP8266-01', 'Sensor de temperatura sala');

-- Tabela de leituras
CREATE TABLE leituras (
    id      INT AUTO_INCREMENT PRIMARY KEY,

    dispositivo_id INT NOT NULL,
    -- Chave estrangeira referenciando a tabela dispositivos

    temperatura DECIMAL(5,2),
    -- DECIMAL(5,2): até 5 dígitos, 2 casas decimais. Ex: 999.99
    -- Mais preciso que FLOAT para valores monetários/medições

    umidade     DECIMAL(5,2),
    rotacao     INT DEFAULT 0,
    estado      VARCHAR(20),
    -- NORMAL, ALERTA, CRITICO, FALHA_SENSOR

    data_hora   DATETIME,

    FOREIGN KEY (dispositivo_id) REFERENCES dispositivos(id)
    -- Garante integridade referencial: não aceita leitura com dispositivo inexistente
);

-- Tabela de alertas
CREATE TABLE alertas (
    id      INT AUTO_INCREMENT PRIMARY KEY,

    leitura_id INT NOT NULL,
    -- Referência à leitura que gerou o alerta

```

```

tipo_alerta VARCHAR(20),
-- ALERTA ou CRITICO

descricao TEXT,
data_hora DATETIME,

FOREIGN KEY (leitura_id) REFERENCES leituras(id)
-- Relacionamento: cada alerta pertence a uma leitura específica
);

-- Criação do usuário da aplicação com permissões limitadas
CREATE USER 'ADM_login'@'localhost' IDENTIFIED BY 'CyberCopilots';
-- '@localhost': só aceita conexões locais (mais seguro)

GRANT SELECT, INSERT, UPDATE, DELETE ON api_automacao.* TO 'ADM_login'@'localhost';
-- Permissões mínimas necessárias (nunca dar GRANT ALL para usuário de aplicação)

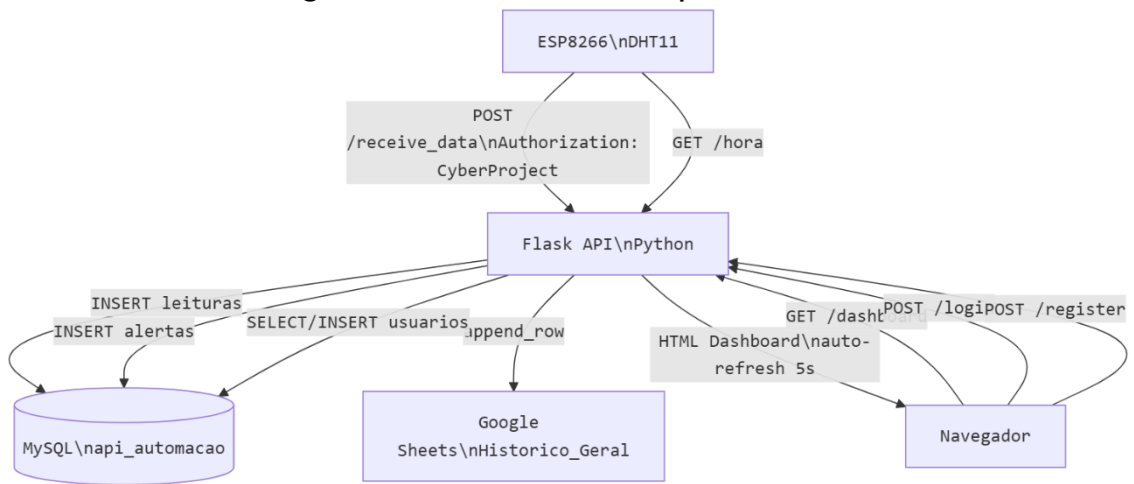
FLUSH PRIVILEGES;
-- Recarrega as permissões no servidor MySQL

```

Perguntas importantes sobre banco:

- *"O que é chave estrangeira?"* → Restrição que garante que o valor em uma coluna existe na tabela referenciada. `leitura_id` em `alertas` deve existir em `leituras.id`.
- *"Por que DECIMAL e não FLOAT para temperatura?"* → FLOAT tem imprecisão binária (22.3 pode virar 22.299999...). DECIMAL armazena exatamente o valor especificado.
- *"O que é bcrypt?"* → Algoritmo de hash com fator de custo ajustável — quanto maior, mais lento de calcular, tornando ataques de força bruta impraticais.

SESSÃO 6 — Diagrama do sistema completo



SESSÃO 7 — Perguntas de professor mais prováveis

Sobre segurança:

- *"Por que não salvar a senha diretamente?"* → Se o banco vazar, senhas em texto puro comprometem os usuários em todos os sistemas onde usam a mesma senha. O hash é irreversível.
- *"O sistema tem proteção contra SQL Injection?"* → Sim, os parâmetros %s com `cursor.execute(query, (valores,))` usam prepared statements implícitos.

Sobre protocolo HTTP:

- *"Qual a diferença entre GET e POST?"* → GET busca dados (idempotente, pode ser cacheado, parâmetros na URL). POST envia dados para criar/processar (não idempotente, corpo na requisição).
- *"O que significa o código 201?"* → Created — padrão REST para indicar que um recurso foi criado com sucesso.

Sobre o ESP8266:

- *"Por que usar `millis()` em vez de `delay()`?"* → `delay()` bloqueia tudo. `millis()` permite que o loop continue processando botões e o servidor HTTP enquanto aguarda o intervalo.
- *"O ESP8266 tem relógio em tempo real?"* → Não. Por isso consulta `/hora` na API Flask para obter o timestamp correto.

Sobre a arquitetura:

- *"O que é uma API REST?"* → Interface de comunicação via HTTP onde cada URL (endpoint) representa um recurso e os métodos HTTP (GET, POST) definem a operação.
- *"Por que salvar em dois lugares (MySQL e Sheets)?"* → MySQL para dados estruturados e consultas complexas. Google Sheets para visualização fácil e compartilhamento com não-técnicos.